

第4章

汇编语言源程序结构

占用内存空间少, 执行速度快

例. 编程实现 $c=a+b$ 并在屏幕上显示出结果

DATA SEGMENT

a DB ?

b DB ?

c DB ?

} 定义字节型变量

string DB 'c=' ; 定义字符串 "c=", 用于输出提示

DATA ENDS

CODE SEGMENT

MAIN PROC FAR

ASSUME CS:CODE, DS:DATA, ES:DATA

START:

[通过 RET 给 CS]

PUSH DS ; 构造返回地址的段地址部分 LEA DX, string ; 把 string 的地址放到 DX 寄存器

SUB AX, AX ; (通过 RET 给 IP)

MOV AH, 09H ; 输出字符串

PUSH AX ; 构造返回地址的偏移地址部分

INT 21H ; 输出 "c="

不能直接给 DS 赋值, 要用 AX 中转

MOV AX, DATA

ADD C, 30H ; 数字转 ASCII 码

MOV DS, AX

} 使 DS, ES 指向数据段
这样才能

MOV DL, C ; 把 '3' 放到 DL 寄存器

MOV ES, AX

MOV AH, 02H ; 输出单个字符

MOV a, 01H

INT 21H ; 输出 '3'

MOV b, 02H

MOV DL, 0AH

MOV AL, a

INT 21H ; 输出换行

ADD AL, b

MOV DL, 0DH

MOV C, AL

INT 21H ; 输出回车

RET

; 程序返回 DOS

/MAIN ENDP

CODE ENDS

END START

用汇编语言编写的源程序在输入计算机后,需将其翻译成目标程序,计算机才能执行相应指令,这个翻译过程称为汇编,完成汇编任务的程序称为汇编程序

汇编程序的分类 { 基本汇编 ASM
宏汇编 MASM

汇编程序以汇编语言源程序文件作为输入,并由它产生2种输出文件 { 目标程序文件
源程序列表文件

过程:

编辑程序 (EDIT) → .ASM文件 → 汇编程序 (MASM) → .OBJ文件 → 连接程序 (LINK) → .EXE文件

计算机上运行汇编语言程序的步骤:

- ① 用编辑程序建立.ASM文件
- ② 用MASM程序把.ASM文件转换成.OBJ文件
- ③ 用LINK程序把.OBJ文件转换成.EXE文件
- ④ 用DOS命令直接键入文件名,执行该程序

汇编语言语句的构成

[名字] 操作符 [操作数] [;注释]
一个符号,表明本条语句的符号地址
不能以数字开头,最多由31个字母、数字及规定的特殊符号组成
机器指令/伪指令/宏指令的助记符
用自定义的标识符,一经使用便具有属性
(1) 段、偏移地址
(2) 类型:变量名的类型可以是BYTE、WORD、DWORD等;标号、段名、子程序名的类型可以是NEAR、FAR(分别表示段内、段间调用)
标号:反映硬指令位置(逻辑地址)和属性的标识符,后跟:分隔
变量:反映伪指令位置(逻辑地址)和属性的标识符,后跟空格或制表符分隔,无:
一个源程序中,每个标识符的定义唯一,但不能是汇编语言采用的保留字

保留字是汇编程序已经利用的标识符(关键字)

{ 硬指令助记符 : MOV, ADD, ...
伪指令助记符 : DB, DW, ...
操作符 : OFFSET, PTR, ...
寄存器名 : AX, CS, ...
预定义符号 : @data

变量

指内存单元的数据,需事先定义才可使用

变量定义为指令(Define)为变量申请固定长度为单位的存储空间,并可同时将相应存储单元初始化

定义后的变量可以利用变量名等方法引用其中的数据

程序代码中通过变量名引用指向的首个数据,通过变量名加减位移量存取以首个数据为基地址的数据

指令

使CPU产生动作、并在程序执行时才处理的指令

区别?

(1) 硬指令

与具体处理器有关而与汇编程序无关

(2) 伪指令

不产生CPU动作、在程序执行前由汇编程序处理的说明性指令

与具体处理器无关而与汇编程序有关

硬指令、伪指令采用易于记忆的符号—助记符来表达

操作数

(1) 常数

由10(D), 16(H), 2(B), 8(O或Q)进制形式表达的数值

默认不加后缀字母的是十进制数

(2) 字符串常量

由' '或" "括起来的单个或多个字符

数值为每个字符对应的ASCII码值

'd' 64H

'AB' 4142H

'Hello, assembly!'

表达式操作符

(1) 算术操作符 + - * / MOD

VIDEO-BUF DB 25*80*2 DUP(?) ↗ 25行*80列*2字节

ARRAY DW 1, 2, 3, 4, 5, 6, 7

ARYEND DW ?

MOV CX, (ARYEND-ARRAY)/2 ↖ 地址差 ↗ 偏移地址
符号地址间减法√ 符号地址±常数√
符号地址间加法× 符号地址乘除×

ADD AX, ARRAY + (6-1)*2 b

MOV AX, BX+1 × 非法, 无法确定是地址还是数值, 切不可直接作操作数

MOV AX, [BX+1]

寄存器间接寻址, 以BX+1为偏移地址取内存中对应的值

(2) 逻辑和移位操作符 AND OR XOR NOT SHL SHR

OPR1 EQU 25 ; 00011001B

OPR2 EQU 7 ; 00000111B

AND AX, OPR1 AND OPR2 ; AND AX, 1

MOV AX, 0FFFFH SHL 2 ; MOV AX, 0FFFC4H

端口读写指令 读 IN - AL, PORT-VAL 端口号别名, 设为 61H; 从 61H 端口读取 1 字节数据存入 AL 寄存器
写 OUT PORT-VAL AND 0FEH, AL ; 将 AL 寄存器中 1 字节数据写入 60H 端口

$$\begin{array}{r} 01100001 \\ \wedge 11111110 \\ \hline 01100000 \rightarrow 60H \end{array}$$

(3) 关系操作符 EQ NE LT LE GT GE

计算结果为逻辑值: 真 0FFFFH (-1)

假 0000H (0)

取变量在数据段中的偏移地址

MOV FID, (OFFSET Y - OFFSET X) LE 128

若 ≤ 128 (真), 结果为 MOV FID, -1

若 > 128 (假), 结果为 MOV FID, 0

(4) 数值回送操作符 OFFSET SEG TYPE LENGTH SIZE

OFFSET/SEG 变量/标号

回送变量或标号的偏移地址/段地址
1单元 = 2字节存储单元

TYPE 变量/标号/常数

DB	<u>DW</u>	DD	DF	DQ	DT	NEAR	FAR	常数
1	2	4	6	8	10	-1	-2	0

LENGTH 变量

回送由 DUP 定义的变量的单元数, 其它情况回送 1

SIZE 变量

LENGTH * (TYPE) 每个单元占多少字节

回送变量的总字节数

ARRAY DW 100 DUP(?) ; 100个未初始化的字单元(每个占2字节)
TABLE DB 'ABCD' ; 字符串变量, 包含4个字节单元(每个占1字节)

ADD SI, TYPE ARRAY ; ADD SI, 2

ADD SI, TYPE TABLE ; ADD SI, 1

MOV CX, LENGTH ARRAY ; MOV CX, 100

MOV CX, LENGTH TABLE ; 未用 DUP, 单元数默认是 1 MOV CX, 1

MOV CX, SIZE ARRAY ; MOV CX, 200

MOV CX, SIZE TABLE ; 未用 DUP, SIZE 默认是单元数为1时的字节数

(5) 属性操作符 PTR 段操作符 SHORT THIS HIGH LOW HIGHWORD LOWWORD MOV CX, 1

类型 PTR 表达式

指定操作数类型(指定BXJ对应内存是字"型")
MOV WORD PTR [BX], 5 ; 将数值5以"2字节"存入 BX 所指内存单元

段操作符

强制指定操作数的段
MOV ES: [BX], AL ; 将AL中的值存入 ES 段中 BX 所指内存单元

SHORT 标号

指定为短跳转(-128 ~ +127)
JMP SHORT NEXT

THIS 类型

别名

给当前地址指定新符号和类型

TA EQU THIS BYTE ; 但不会给TA分配任何字节的内存
TD DW 1234H ; 将TD变量分配到刚记录的这地址中
NEXT EQU THIS FAR ; 给当前地址定义远标号NEXT, 可以跨段跳转
MOV AX, 2

HIGH 和 LOW

等价
CONS EQU 1234H

MOV AH, HIGH CONS ; 从16位常数中取出高8位

MOV AL, LOW CONS ; 从16位常数中取出低8位

伪指令 (伪操作)

伪操作是汇编程序对源程序进行汇编时处理的操作, 完成处理器选择、存储模式定义、数据定义、存储器分配、指示程序开始结束等功能

(1) 数据定义及存储器分配伪指令

[变量] 助记符 表达式[, 表达式, ...]

DB DW DD DF DQ DT

DATA-BYTE →	0AH
	04H
	10H
	-
DATA-WORD →	64H
	00H
	00H
	01H
	FBH
	FFH
	-
	-

ASCII码:

A = 65D = 41H

a = 97D = 61H

DATA-BYTE DB 10, 4, 10H, ?

DATA-WORD DW 100, 100H, -5, ?

100000 ... 0101

→ 11111111 1010

→ 11111111 1011

FFB B

ARRAY DB 'HELLO'

DB 'AB'

DW 'AB'

ARRAY →	48H
	45H
	4CH
	4CH
	4FH
	41H
	42H
	42H
	41H

PAR1 DW 100, 200

PAR2 DW 300, 400

ADDR-TABLE DW PAR1, PAR2 ; 存 PAR1, PAR2 的偏移地址

VAR DB 100 DUP(?) ; 定义字节型变量VAR, 分配100个未初始化的字节

DB 2 DUP(0, 2 DUP(1, 2), 3) ; 给VAR追加12字节的初始化数据:

0, 1, 2, 1, 2, 3, 0, 1, 2, 1, 2, 3

OPER1 DB ?, ?

OPER2 DW ?, ?

:

MOV OPER1, 0

MOV OPER2, 0

OPER1 DB 1, 2

OPER2 DW 1234H, 5678H

:

MOV ^字AX, ^{字节}OPER1+1

MOV ^{字节}AL, ^字OPER2

MOV AX, WORD PTR OPER1+1 ✓

MOV AL, ^{字节→字}BYTE PTR OPER2 ✓

/ 字→字节

PTR, OFFSET... 不是伪指令而是操作符

伪指令DB后面可以是:

字节常量、不确定常量(?), 重复句(DUP), 串常量及其任意组合

伪指令DW后面可以是:

字常量、?, DUP、地址表达式(偏移地址), 1/2个字符组成的串常量及其任意组合

伪指令DD后面可以是:

双字常量、?, DUP、地址表达式(偏移地址对应低地址, 段地址对应高地址存入存储器), 1~4个字符组成的串常量及其任意组合

(2) 符号定义伪指令

1) EQU

标识符 EQU 表达式

用表达式来定义标识符,即使得标识符等同表达式

COUNT EQU 8

; 定义符号常量 COUNT, 使其等同于 8

ADDR EQU SS:[BX][ESI]

; 使 ADDR 等同于地址表达式 SS:[BX][ESI]

$PA = SS \text{ 段地址} + BX + SI$

基址变址寻址

2) =

与 EQU 类似, 但 = 可对同一标识符重新定义

COUNT = 8

; 定义 COUNT 等于 7

COUNT = COUNT + 1

; 重新定义 COUNT 等于 8

3) LABEL

标识符 LABEL 类型

定义标识符的类型

若标识符是变量, 则类型可以是 BYTE、WORD、DWORD
若标识符是标号, 则类型可以是 FAR、NEAR

BYTE-ARRAY LABEL BYTE

WORD-ARRAY DW 50 DUP(?)

(3) 段定义伪指令

指示汇编程序如何按段组织、使用寄存器

1) SEGMENT 与 END

不能把相同类别的段合并成一个段,但在连接后形成的装入模块中可以把它们的位置靠在一起

段名 SEGMENT [定位类型][组合类型][类别]

111

段名 ENDS

2) ASSUMĒ

ASSUME 段寄存器名: 段名/组名 [, 段寄存器名: 段名/组名]

通知汇编程序 CS、DS、SS、ES 被设定为哪些段的段地址寄存器，
进而在汇编时能知道语句中引用的变量、标号、表达式对应的段

ASSUME [段寄存器:] NOTHING : 取消由ASSUME指定的段寄存器与段组的联系

3) GROUP

组名 GROUP 段名[,...]

把模块中若干不同的段集合成一个组并赋组名,使它们装在同一物理段中(64KB)

4) DRG

ORG 表达式 } 常量
ORG \$ + 表达式

使汇编程序中其后的指令或数据从表达式的值指定的偏移地址开始存放

5) 程序计数器

汇编程序汇编源程序时,使用地址计数器保存当前正在汇编指令的偏移地址

当开始汇编或每一段开始时,将地址计数器初始化为0,此后每处理1条指令就增加1个值,该值为指令所需字节数

地址计数器的值用\$表示,汇编语言允许用户直接用\$引用地址计数器的值

保存当前正在汇编的指令的地址

0074HARRY

ORG \$+8 ; 跳过8个字节的存储区

JNE \$+6 ; 满足条件则转向JNE的首地址+6字节处

JMP \$+2; 转向下条指令

JMP指令本身占2字节

⌘用在伪操作的参数字段

表示地址计数器的当前值

ARRAY DW 1, 2, \$+4, 3, 4, \$+4

上	
→	01H
	00H
	02H
	00H
	7CH
	00H
	03H
	00H
	04H
	00H
	82H
	00H

(4) 过程定义伪指令

FAR: 远过程, 段间调用

过程名 PROC [类型] / NEAR: 近过程, 段内调用 (缺省, 默认)

:

RET

过程名 ENDP ; 过程结束

(5) 汇编结束伪指令 END

→ 启动地址 如 START

END [表达式]

★ 任何一个完整的源程序均应有END指令

汇编语言源程序格式

汇编语言源程序的分段结构(逻辑分段)

代码段、数据段、堆栈段、附加段

汇编语言源程序的语句类型

指令语句、伪指令语句、宏指令语句

完整的汇编语言源程序由段组成:

一个汇编语言源程序可以包含若干个CS、DS、SS、ES, 段间顺序无规定

- 要独立运行的程序必须包含一个代码段并指示程序执行的起始点, 一个程序只有一个起始点
所有可执行语句必须位于某一代码段内, 说明性语句可根据需要位于任一段内
- 采用 START 标识符作为标号指明程序开始执行的位置,
链接程序会根据程序起始点正确设置 CS 和 IP 值,

根据程序大小和堆栈段大小设置 SS 和 SP 值

但链接程序没有设置 DS 和 ES 值, 若使用 DS 段或 ES 段则必须明确给 DS 或 ES 赋值

START:

MOV AX, DATA;

MOV DX, AX;

⋮

★ 大多程序需要数据段

程序
终
止

- 应用程序执行结束, 应将控制权交还操作系统

返回DOS的方法:

一般利用DOS功能调用的4CH子功能实现,

需要的入口参数是 AL = 返回数码 (0 表示程序无误)

↓

应用程序终止:

MOV AX, 4C00H

INT 21H

汇
编
结
束

- 汇编结束表示汇编程序到此结束将源程序翻译成目标模块代码的过程
源程序最后必须有一条END伪指令

END [标号]

- 可选的“标号”参数指定程序开始执行点, 链接程序据此设置CS和IP值

例. 屏幕显示字符串并自动退出

DATA SEGMENT

STRING DB 'Hello, assembly!', 0DH, 0AH, '\$'

DATA ENDS

CODE SEGMENT

ASSUME DS:DATA, CS:CODE

START:

MOV AX, DATA

MOV DS, AX

MOV DX, OFFSET STRING

MOV AH, 09H

INT 21H

MOV AX, 4C00H

INT 21H

CODE ENDS

END START

1. 使用DOS系统功能调用时, 使用的软中断指令是 INT 21H

2. 伪指令 OFFSET 表达的含义是回送变量或标号的 (B)

A. 段地址值 ☒ B. 偏移地址值 C. 物理地址值 D. 操作数

3. 表示 一条指令的存储单元的符号地址 称为 (A)

☒ A. 标号 B. 变量 C. 偏移量 D. 类型

4. 在一连续存储单元, 依次存放数据 41H, 42H, ..., 46H, 数据定义语句可为 (D)

☒ A. DB 41, 42, 43, 44, 45, 46 → 默认10进制

☒ B. DW 4142H, 4344H, 4546H →

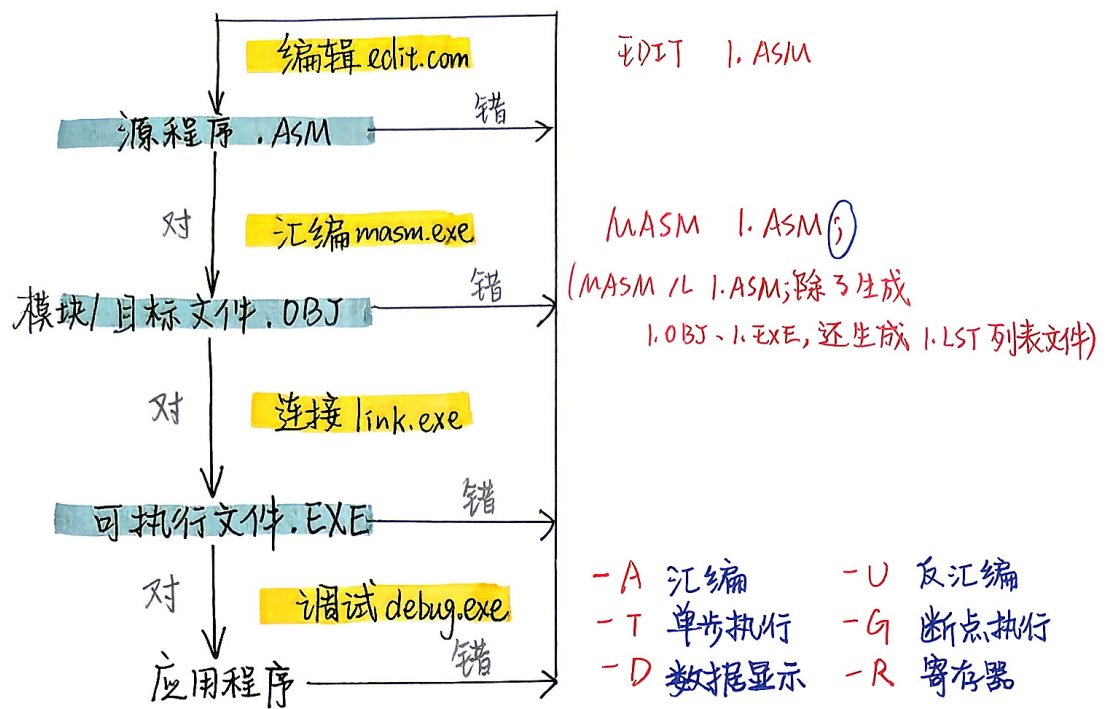
☒ C. DW 'AB', 'CD', 'EF' →

☒ D. DW 'BA', 'DC', 'FE'

41H
42H
43H
44H
⋮

42H
41H
44H
43H
⋮

汇编语言的上机过程



DOS 支持 2 种可执行程序结构：

① EXE 程序

可有多段代码段、数据段
程序长度可超过 64KB

② COM 程序

仅一个逻辑段
程序长度不超过 64KB

逐步求精的、模块化的设计方法

作为过程和宏实现

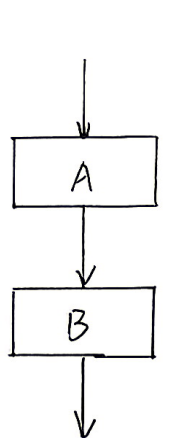
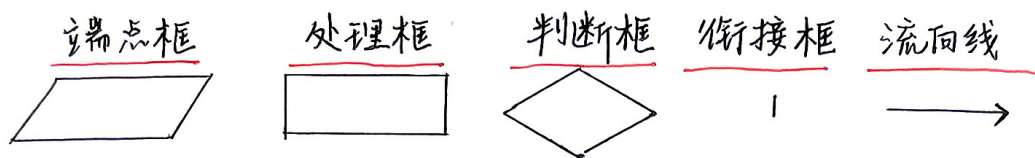
一个良好程序的标准：可读性强、执行时间短、占用内存少、语句行数少

流程图应当是逻辑的而不是物理的

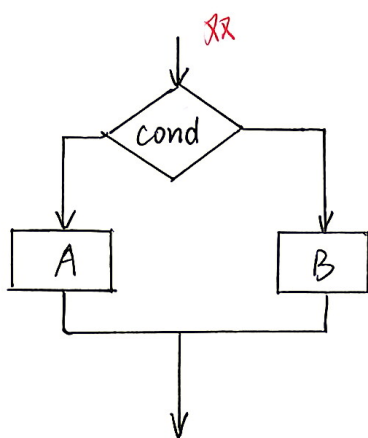
✓

✗

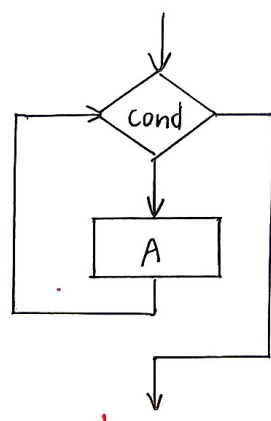
无论在何机器使用何语言，同一问题同一算法的流程图应唯一



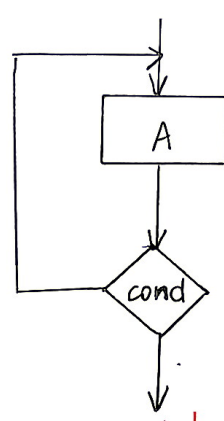
顺序结构



分支结构

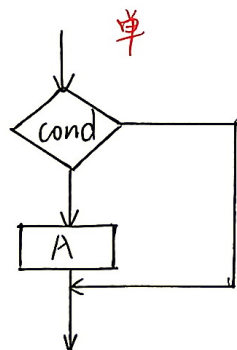


while

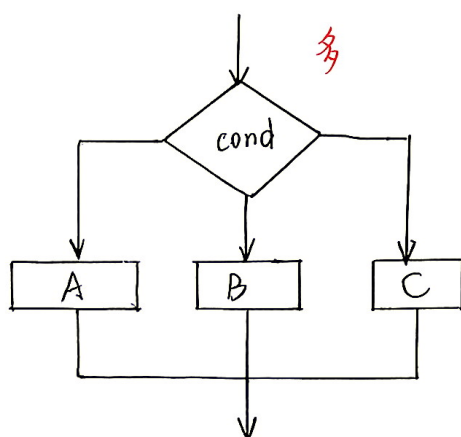


until

循环结构



单



多

顺序程序设计

程序从开始执行到结束一直按指令序列在存储器中的存放顺序来执行

执行顺序由 CS:IP 提供的指令地址来控制, 每次修改 IP 的内容指向下一条指令

例. 写程序计算 $Z = (3X + Y - 5) / 2$

数据段

```
DATA SEGMENT
    VARX DW 15
    VARY DW 10
    VARZ DW ?
DATA ENDS
```

堆栈段

```
STACK1 SEGMENT PARA STACK
    DW 20H DUP(0) ; 分配 2x20H=64 字节堆栈空间, 初始值为 0
STACK1 ENDS
```

代码段

```
CODE SEGMENT
    ASSUME CS:CODE, DS:DATA, SS:STACK1
    COMP PROC FAR
    START:
        PUSH DS
        MOV AX, 0
        PUSH AX ; 保护返回DOS的地址
        MOV AX, DATA
        MOV DS, AX ; 初始化DS
        MOV AX, VARX ; AX=VARX
        SHL AX, 1 ; AX=2VARX
        ADD AX, VARY ; AX=3VARX
        ADD AX, VARY ; AX=3VARX+VARY
        SUB AX, 5 ; AX=3VARX+VARY-5
        SAR AX, 1 ; AX=(3VARX+VARY-5)/2
        MOV VARZ, AX ; VARZ=(3VARX+VARY-5)/2
        RET
    COMP ENDP
CODE ENDS
END START
```

例. 利用学号查该学生的数学成绩. 首先在数据段中建立一个成绩表TABLE, 在表中按学号从小到大的顺序存放分数. 要查的学号存放在变量NUM中

```
DATA SEGMENT
    TABLE DB 98, 99, 100, 97, 98, 99, 100, 100, 100, 100 ; 表 (依学号次序对应)
    NUM DB 8 ; 要查的学号
    MATH DB ? ; 该生数学成绩
DATA ENDS

STACK SEGMENT PARA STACK
    DW 20H DUP(0)
STACK ENDS

CODE SEGMENT
    ASSUME CS: CODE, DS: DATA, SS: STACK
    START:
        MOV AX, DATA
        MOV DS, AX
        ; 取TABLE偏移地址
        MOV BX, OFFSET TABLE ; BX指向TABLE首地址
        XOR AH, AH ; AH清零
        ; 位数匹配才可赋值
        MOV AL, NUM ; AL = 学号
        DEC AL ; 得索引序号 (学号-1)
        ; XLAT TABLE
        ADD BX, AX ; 目标学号的分数的地址 = 首地址 + 索引序号
        MOV AL, [BX] ; BX指向TABLE[]的内容, 即分数, 给AL (位数匹配)
        MOV MATH, AL ; 结果存入MATH
        ; 返回DOS
        MOV AH, 4CH
        INT 21H
    CODE ENDS
    END START
```

分支程序设计

2种方法 { 选用条件转移指令 : 测试法分支程序设计
 { 选用间接寻址无条件转移 : 跳跃表法分支程序设计

- 分支程序依条件真假决定是否执行
- 判断的条件是CMP TEST等执行后形成的状态标志
- 转移指令Jcc和JMP可实现分支控制

例. 已知内存中有一个字节单元NUM, 存有带符号数据, 要求计算出它的绝对值后放入RESULT单元

DATA SEGMENT

NUM DB -25

RESULT DB ?

DATA ENDS

CODE SEGMENT

ASSUME DS: DATA, CS: CODE

START:

MOV AX, DATA

MOV DS, AX

MOV AL, NUM

TEST AL, 80H ; AL与80H按位与, 保留了最高位

JZ NEXT ; 等于0 (ZF=1) 则跳至NEXT, 否则说明是负数

NEG AL ; 负数要求补才得绝对值

NEXT:

MOV RESULT, AL

MOV AH, 4CH

INT 21H

CODE ENDS

END START

例. 试编一程序, 求3个带符号字数据中的最大值, 并将最大值存入MAX字单元中.

设3个带符号数分别在3个字变量X、Y、Z中存储

```
DATA SEGMENT
    X DW -0ABH
    Y DW 205
    Z DW 200
    MAX DW ?
DATA ENDS
```

```
CODE SEGMENT
    ASSUME DS:DATA, CS:CODE
    START:
        MOV AX, DATA
        MOV DS, AX


---


        MOV AX, X
        CMP AX, Y
        JGE LI
        MOV AX, Y

    LI:
        CMP AX, Z
        JGE EXIT
        MOV AX, Z

    EXIT:
        MOV MAX, AX


---


        MOV AH, 4CH
        INT 21H
CODE ENDS
END START
```

例. 设某程序有8跳分支, 试根据给定的N值(1~8)将程序的执行转移到其中的一路分支

DATA SEGMENT

TABLE DW P1, P2, P3, P4, P5, P6, P7, P8

NUM DB 5

DATA ENDS

CODE SEGMENT

ASSUME DS:DATA, CS:CODE

START:

MOV AX, DATA

MOV DS, AX

MOV AL, NUM

DEC AL ; NUM-1

ADD AL, AL ; (NUM-1) * 2

MOV BL, AL

MOV BH, 0

JMP TABLE[BX] ; 查表地址 = TABLE + (NUM-1) * 2

P1:

...

JMP EXIT

:

P7:

...

JMP EXIT

P8:

...

EXIT:

MOV AH, 4CH

INT 21H

CODE ENDS

END START

例. 根据 AL 寄存器中哪一位为 1 (从低位到高位), 把程序转移到 8 个不同的程序分支

DATA SEGMENT

BRANCH-TABLE DW routine1, routine2, ..., routine8

DATA ENDS

CODE SEGMENT

ASSUME CS: CODE, DS: DATA

START:

MOV AX, DATA

MOV DS, AX

CMP AL, 0

JE CONTINUE

MOV SI, 0

LEA BX, BRANCH-TABLE ; 使 BX 指向分支表的首地址

ADD BX, 7 * TYPE BRANCH-TABLE MOV CX, 8

L:

SHR AL, 1 ; 最低位移到 CF

JNC ADDI^{SUBI}; CF=0 (最低位为 0) 则跳转

JMP BRANCH-TABLE[SI] JMP WORD PTR [BX] ; 终于最低位为 1, 跳转到 BX 指向的分支

ADDI^{SUBI}:

ADD SI, TYPE BRANCH-TABLE; ADD^{SUB} BX, TYPE BRANCH-TABLE ; BX+=2, BX 往下挪 2 个内存单元 (表元素是字)

JMP L

LOOP L (避免死循环)

routine1:

MOV DL, '1'

JMP SHOW

routine2:

MOV DL, '2'

JMP SHOW

⋮

routine8:

MOV DL, '8'

SHOW:

MOV AH, 02H

INT 21H

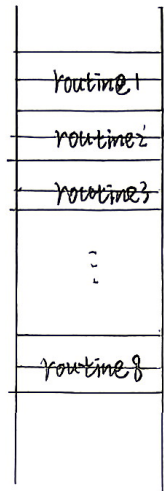
CONTINUE:

MOV AH, 4CH

INT 21H

CODE ENDS

END START



从高位到低位

; 显示单个字符

寄存器间接寻址

寄存器相对寻址

基址变址寻址:

CMP AL, 0

JE CONTINUE

LEA BX, BRANCH-TABLE

MOV SI, 7 * TYPE BRANCH-TABLE

MOV CX, 8

L:

SHL AL, 1

JNC SUBI

JMP WORD PTR [BX][SI]

SUBI:

SUB SI, TYPE BRANCH-TABLE

LOOP L

循环程序设计

3个组成部分 = { 循环初始化部分
循环体
循环控制部分 } 循环结构程序设计的关键

循环控制的方法: { 计数控制法 (循环次数已知)
条件控制法 (循环次数未知)
混合控制法 (已知循环次数和循环继续的条件)

(1) 计数控制法

1) 重复 N-1 至 0 结束

```
MOV CX, N
:
LABEL:
:
DEC CX
JNZ LABEL
```

2) 重复 -N+1 至 0 结束

```
MOV CX, -N
:
LABEL:
:
INC CX
JNZ LABEL
```

3) 重复 +1 至 N 结束

```
MOV CX, 0
:
LABEL:
:
INC CX
CMP CX, N
JNZ LABEL
```

例. 已知有 N 个元素存放在以

BUF 为首地址的字节存储区中, 试统计其中正数的个数, 并存放在随后的 NUM 字节单元中

DATA SEGMENT

BUF DB 25, 36, -6, 99, -12, 52, 10

NUM DB 0

DATA ENDS

CODE SEGMENT

ASSUME CS: CODE, DS: DATA

MAIN PROC FAR

START:

PUSH DS

XOR AX, AX

PUSH AX

MOV AX, DATA

MOV DS, AX

MOV CX, NUM-BUF

MOV AX, 0

LEA BX, BUF

MAIN ENDP

CODE ENDS

END START

[BX] 中是当前数组元素

AGAIN: ↑

CMP BYTE PTR [BX], 0

JLE LAB1

INC AX ; 统计正数个数

LAB1:

INC BX

LOOP AGAIN

MOV NUM, AX

RET

例. 统计 AX 寄存器中为 1 的位的个数, 并将统计结果放在 CL 寄存器中

MOV CL, 0 ; CL 清零

MOV BX, 16 ; 循环检查 16 次 (AX 有 16 位)

LAB:

SAL AX, 1 ; 算术左移 1 位, 最高位移至 CF

JNC LAB1 ; CF=0, 不统计

INC CL ; CF=1, 统计 (CL+1)

LAB1:

DEC BX ; BX--

JNZ LAB ; BX 不为 0, 循环次数没到

EXIT:

...

(2) 条件控制循环

例. 统计AX寄存器中为1的位数, 并将统计结果放在CL寄存器中

MOV CL, 0

LAB:

AND AX, AX ; 影响ZF标志位 (若AX为0, 则ZF=1; 反之, ZF=0)

JZ EXIT ; ZF=1 (AX为0) 则跳转至EXIT

SAL AX, 1 ; 将AX内容左移1位, 最高位移到CF

JNC LAB ; CF=0 (这一位是0) 则跳转至LAB

INC CL ; CF=1 则CL+=1

JMP LAB

EXIT:

...

多重循环程序的设计要点:

- 内层循环必须完全包含在外层循环
- 每次通过外层循环进入内层循环时内层循环的初始条件必须重设

例. 有一首地址为A的数组, 编制程序使该数组按从小到大的顺序排列

采用冒泡排序算法, 从第1个数开始依次对相邻两个数进行比较,

小数在前、大数在后, 若次序不对, 则两数交换位置

序号	地址	数	比较遍数			
			1	2	3	4
1	A	32	32	16	15	8
2	A+2	85	16	15	8	15
3	A+4	16	15	8	16	16
4	A+6	15	8	32	32	32
5	A+8	8	85	85	85	85

MOV CX, COUNT ; 将元素个数加载到CX (N个数)

DEC CX ; CX-=1 (N-1遍排序)

LOOP1:

MOV DI, CX ; 将外层循环的CX值传给DI (内层循环比较次数)

MOV BX, 0 ; 使数组偏移地址为0, 从第1个数开始
每一遍比较

LOOP2:

MOV AX, ADDRESS[BX]

CMP AX, ADDRESS[BX+2]

JLE CONTINUE ; 当前元素 ≤ 下元素, 不交换

XCHG AX, ADDRESS[BX+2] ; 交换

MOV ADDRESS[BX], AX ; 换完的值写回ADDRESS[BX]

CONTINUE:

ADD BX, 2 ; 继续往下比较

LOOP LOOP2

LOOP LOOP1

EXIT:

RET

1. 阅读下列程序段, 程序执行到HLT时, (AX) = 0FH 或 15, (BX) = 6.

```

XOR AX, AX      ; 使AX清零
MOV BX, 1       ; BX=1
MOV CX, 5       ; CX=5
LOOP1:
  ADD AX, BX     CX=5  4  3  2  1
                  AX=1  AX=1+2 AX=(1+2)+3 AX=(1+2+3)+4 AX=(1+2+3+4)+5
  INC BX         BX=2  BX=3  BX=4  BX=5  BX=6
  LOOP LOOP1     ; CX--1, CX不为0就返回到LOOP1
                  ; CX=0
HLT

```

2. 试分析下列程序段, 程序执行后, NUM2 字节单元的内容是? 若 NUM1 字节单元的值为 39H, 运行后, NUM2 字节单元的内容是? 程序完成何操作?

```

DATA SEGMENT
  NUM1 DB 41H 'A'
  NUM2 DB ?
DATA ENDS

```

```

CODE SEGMENT
  ASSUME CS:CODE, DS:DATA
  START:

```

'0' ... '9' ... 'A' ... 'F'
30H ... 39H ... 41H ... 46H

```

  MOV AX, DATA

```

```

  MOV DS, AX

```

```

  MOV AH, NUM1

```

; 将待转换的ASCII字符加载到寄存器
; 若 AH > 39H, 说明是 'A' ~ 'F'; 否则, 说明是 '0' ~ '9'

```

  CMP AH, 39H

```

```

  JBE NEXT

```

; '0' ~ '9' 则跳转至 NEXT

```

  SUB AH, 7

```

; 'A' ~ 'F' 则还要 "-7"

```

  NEXT:

```

```

  SUB AH, 30H

```

; 得数值

```

  MOV NUM2, AH

```

```

  MOV AH, 4CH

```

① 3AH

```

  INT 21H

```

② 09H

```

CODE ENDS

```

③ 该程序可将 NUM1 字节单元中的 ASCII 字符

```

END START

```

'0' ~ '9', 'A' ~ 'F'

转换成十六进制数 00H ~ 09H, 0AH ~ 0FH

3. 下列程序完成将 BX 寄存器的数值以十六进制显示输出, 完成填空

CODE SEGMENT

ASSUME CS: CODE

START:

MOV BX, 0DA12H

MOV CH, 4

ROTATE: → 循环处理每4位二进制, 也就是1位十六进制

MOV CL, 4

ROL BX, CL ; "循环左移4位, 低4位移到"原高4位", 高4位移到"原低4位"

MOV AL, BL > 将BL中的8位存入AL中, 提取低4位, 也就是"原高4位"

AND AL, 0FH

ADD AL, 30H ; 将这位十六进制数转换为ASCII码 (处理34位二进制位)

CMP AL, 3AH

JL PRINTIT > 比3AH小

ADD AL, 07H 3AH, 就再加07H

PRINTIT:

MOV DL, AL

MOV AH, 02H 字符输出到屏幕

INT 21H

DEC CH

JNZ ROTATE > 还有16进制数没处理 (CH ≠ 0), 就继续回去. 循环

MOV AH, 4CH

INT 21H

CODE ENDS

END START

子程序设计

子程序又称为过程，调用子程序的程序称为主程序

效果：

简化源程序结构、节省目标程序存储空间、提高程序设计效率

定义 (由伪指令 PROC、ENDP 实现)

过程名 PROC [NEAR 或 FAR]

过程名 ENDP

NEAR: 只能被相同代码段的其他程序调用

FAR: 可被不同代码段的程序调用

- 若子程序和主程序在同一代码段，则子程序定义为 NEAR 属性
- 若子程序和主程序不在同一代码段，则子程序定义为 FAR 属性
- 主程序通常定义为 FAR 属性 (由于主程序被看作 DOS 调用的一个子程序，以便执行完返回 DOS)
- 子程序的位置可以在代码段主程序开始执行之前，也可以在代码段末尾

主程序执行终止后的位置

例. 编写一个子程序，键入一位十进制数 (子程序名: stdin, 功能: 完成从键盘输入一位十进制数
入口参数: 等待键盘输入, 出口参数: AL 中存放输入的数值)

STDIN PROC

MOV AH, 01H

INT 21H

CMP AL, 30H

JL NEXT

CMP AL, 39H

JG NEXT

AND AL, 0FH

NEXT:

RET

STDIN ENDP

> 从键盘读取输入的字符，其 ASCII 码存入 AL

; 与字符 '0' 的 ASCII 码相比

; < '0', 则直接返回

; > '0', 则与字符 '9' 的 ASCII 码相比

; > '9', 则直接返回

; ≤ '9' (同时 > '0'), 则把高 4 位置 0, 低 4 位保留,
即可获得数值

调用与返回

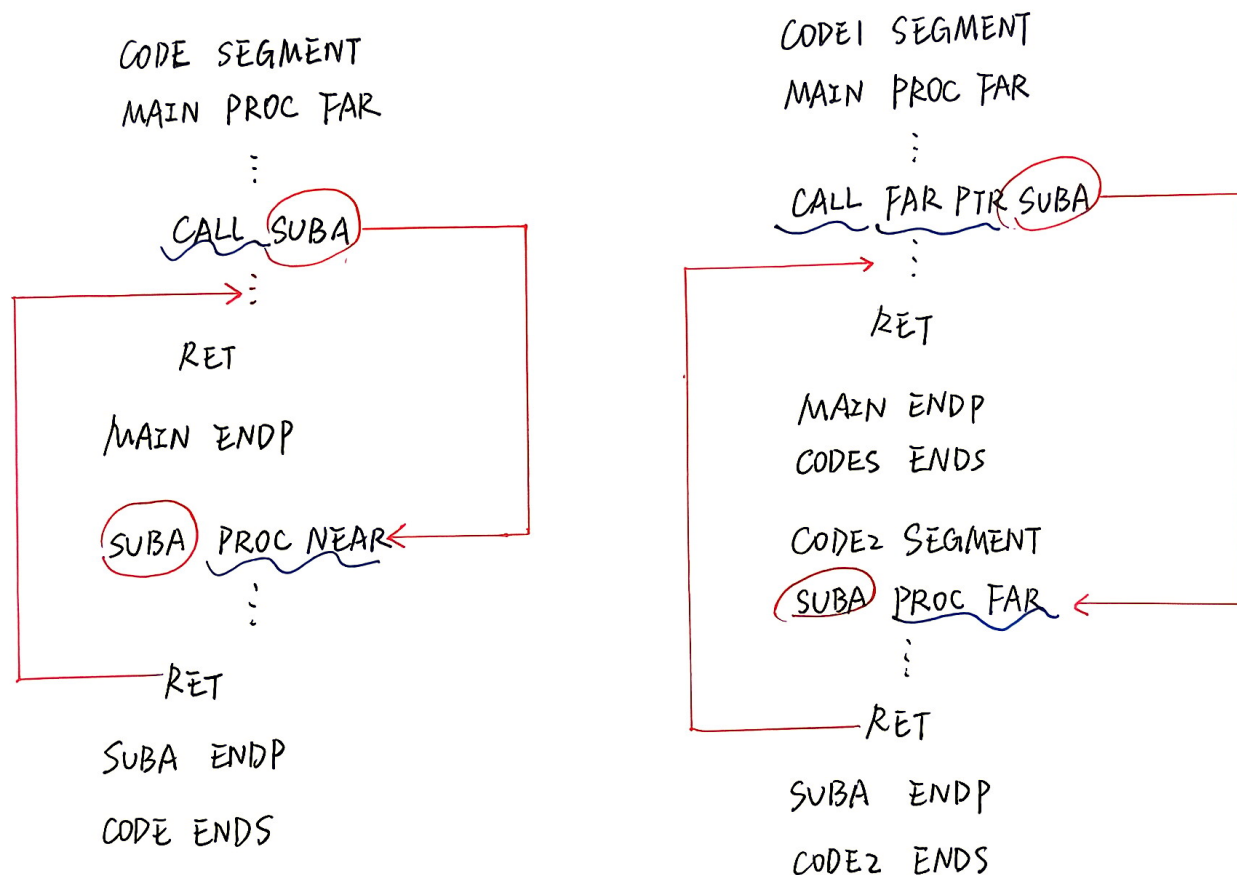
子程序调用与返回由 CALL 和 RET 指令实现

子程序调用指令首先把子程序的返回地址 (CALL 指令的下一条指令的地址) 压入堆栈, 然后转移到子程序的入口地址执行子程序

根据子程序和主程序是否在同一代码段, 分为段内调用和段间调用

子程序返回指令负责把压入栈区的返回地址弹出送 IP 或 CS:IP, 实现返回主程序继续往下执行。

与子程序的段内调用和段间调用相对应, 子程序的返回也分为段内返回和段间返回



保护与恢复

目的与效果:

进入子程序后, 就把它所用到的寄存器内容压进栈, 返回时再把它们弹出栈

避免主程序与子程序使用相同寄存器时修改内容导致出错

例. 若子程序 PROG 中改变了寄存器 AX, BX, CX, DX 的值, 则可采用如下方法保护和恢复现场

由堆栈“先进后出”的操作特点, 恢复寄存器的顺序不能搞错

```
PROG PROC
```

```
PUSH AX
```

```
PUSH BX
```

```
PUSH CX
```

```
PUSH DX
```

```
⋮
```

```
POP DX
```

```
POP CX
```

```
POP BX
```

```
POP AX
```

```
RET
```

```
PROG ENDP
```

子程序的参数传递

{ 入口参数(输入参数): 主程序调用子程序时, 提供给子程序的参数
出口参数(输出参数): 子程序执行结束返回给主程序的参数

{ 传数值: 传递数据本身
传地址: 传递数据的主存地址

常用的参数传递方法 { 寄存器
存储器
堆栈

(1) 用寄存器传递参数 (适用于传递参数较少)

注意! 带出口参数的寄存器不能保护和恢复

例. 从键盘输入一组字符直到"0"为止. 编一子程序, 完成输入小写字母时修改为大写字母, 输入的字符存放在 String 为首址的存储单元中

DATA SEGMENT

STRING DB 100 DUP(?)

DATA ENDS

CODE SEGMENT

ASSUME CS: CODE, DS: DATA

START:

MOV AX, DATA

MOV DS, AX

MOV DI, OFFSET STRING

AGAIN:

MOV AH, 01H

INT 21H

CMP AL, '0'

JE EXIT

CALL STOB ; 调用子程序

MOV [DI], AL ; 把处理后的AL中的字符存入DI指向的存储单元

INC DI

JMP AGAIN

EXIT:

MOV AH, 4CH

INT 21H

STOB PROC NEAR

CMP AL, 61H ; 比较输入字符与'a'的ASCII码

JNB NEXT ; 不是小写字母则不处理

CMP AL, 7AH ; 比较输入字符与'z'的ASCII码

JAB NEXT ; 不是小写字母则不处理

SUB AL, 20H ; 是小写字母则-20H得大写字母

NEXT:

RET ; 处理后结果存于AL

STOB ENDP

CODE ENDS

END START

(AL) 传递入口参数、出口参数

(2) 用存储单元传递参数 (适用于传递参数较多)

主程序在存储单元建立参数表, 将参数表传递给子程序,

子程序取得所需参数并将结果存放到指定存储单元中

例. 从键盘输入一组字符直到 "0" 为止. 编一子程序, 完成输入小写字母时修改为大写字母, 输入的字符存放在 String 为首址的存储单元中

DATA SEGMENT

STRING DB 100 DUP (?)

LENGTH DW ?

DATA ENDS

CODE SEGMENT

ASSUME CS: CODE, DS: DATA

MAIN PROC FAR

START:

PUSH DS

MOV AX, 0

PUSH AX

MOV AX, DATA

MOV DS, AX

MOV CX, 0

MOV DI, OFFSET STRING

AGAIN:

MOV AH, 01H

INT 21H

CMP AL, '0'

JE NEXT

MOV [DI], AL ; 将键入字符存入 DI 指向的内存单元

INC DI

INC CX ; 统计字符长度

JMP AGAIN

NEXT:

MOV LENGTH, CX ; 将字符长度存入 LENGTH 变量

LEA BX, STRING ; 将 STRING 首地址存入 BX

CALL STOB1

RET

STOB1 PROC

MOV CX, LENGTH ; 将字符长度存入 CX

AGAIN1: ; 它还要变

MOV AL, [BX] ; 从 BX 指向的内存单元读取 1 个字符

CMP AL, 61H > 'a'

JB NEXT1

CMP AL, 7AH > 'z'

JA NEXT1

SUB AL, 20H

MOV [BX], AL ; 转换后的字符存回原内存单元

NEXT1:

INC BX

DEC CX

JNZ AGAIN1 ; 不是 0 则跳转

RET

STOB1 ENDP

MAIN ENDP

CODE ENDS

END START

(3) 用堆栈传递参数 (适用于传递参数较多)

用堆栈传递入口参数时,要在调用子程序前把有关参数依次压栈,

子程序从堆栈中取得入口参数;

用堆栈传递出口参数时,要在子程序返回前把有关参数依次压栈,

主程序从堆栈中取得出口参数

例. 编一子程序,求有符号数组ARRAY中元素的最小值.数组元素的个数存放在COUNT字单元中

DATA SEGMENT

ARRAY DW 100,66,-1,88,20

COUNT DW 5

MIN DW ?

DATA ENDS

STACK SEGMENT

DW 128 DUP(?) ; 传递参数,保护寄存器

STACK ENDS

CODE SEGMENT

ASSUME CS:CODE,DS:DATA,SS:STACK

START:

MOV AX,DATA

MOV DS,AX

MOV AX,OFFSET ARRAY > 数组首地址压栈

PUSH AX

MOV AX,COUNT > 元素个数压栈

PUSH AX

CALL FINDMIN

~~MOV MIN,AX~~

MOV AH,4CH

INT 21H

FINDMIN PROC

PUSH BP

MOV BP,SP ; BP指向当前栈顶

PUSH BX ; [BP+2]是CALL自动压入的返回地址

PUSH CX

MOV BX,[BP+6] ; 取数组首地址

MOV CX,[BP+4] ; 取元素个数

DEC CX

MOV AX,[BX] ; AX为数组第1个元素

AGAIN:

ADD BX,2

CMP [BX],AX

~~JG~~ NEXT

MOV AX,[BX]

NEXT:

LOOP AGAIN

POP CX

POP BX

POP BP

RET

FINDMIN ENDP

CODE ENDS

END START

子程序嵌套与递归

一个子程序可作为调用程序去调用另一个子程序,称为子程序嵌套

递归子程序: 一个子程序调用的是子程序本身

例. 计算 $n!$ ($0 \leq n \leq 6$)
$$n! = \begin{cases} 1 & n=0 \\ n \times (n-1)! & n>0 \end{cases}$$

```
DATA SEGMENT
    NUM DB 3
    RESULT DW ?
```

```
DATA ENDS
```

```
STACK SEGMENT PARA STACK 'STACK'
    BP 100 DUP (?)
STACK ENDS
```

```
CODE SEGMENT
```

```
    ASSUME CS:CODE, DS:DATA, SS:STACK
```

```
    MAIN PROC FAR
```

```
    BEGIN:
```

```
        PUSH DS
```

```
        XOR AX, AX
```

```
        PUSH AX
```

```
        MOV AX, DATA
```

```
        MOV DS, AX
```

```
        MOV AH, 0H
```

```
        MOV AL, NUM
```

```
        CALL FACTOR
```

```
        MOV RESULT, AX
```

```
        RET
```

```
    MAIN ENDP
```

```
FACTOR PROC
```

```
    PUSH AX    3, 2, 1 依次入栈
```

```
    SUB AX, 1
```

```
    JNE F-COUNT 不为0就继续递归
```

```
    POP AX    ; AX=1
```

```
    JMP RETURN
```

```
F-COUNT:
```

```
    CALL FACTOR    CX=2 3
```

```
    POP CX    ; 弹出堆栈顶部的值到CX
```

```
    MUL CL    ; AX=AX*CL  
                1*2 *3
```

```
    RETURN
```

```
    RET    ; RET回到最近一次CALL压入的  
            地址的下一条指令
```

```
FACTOR ENDP
```

```
CODE ENDS
```

```
END BEGIN
```

PUSH: 把寄存器的值存到堆栈顶部

POP: 从堆栈顶部取值到寄存器

子程序库

模块化的程序设计：

对各个子程序模块单独汇编产生相应的目标模块(.obj文件)，再用

连接程序把它们连起来，形成完整的可执行程序 结构清晰 便于调试

各模块之间存在相互调用，即一个模块会引用在另一个模块中定义的标识符

变量、标号、过程名等

标识符有2种：

(1) 在本模块中定义供本模块使用的：局部标识符

(2) 在一个模块中定义而在另一个模块中引用的：外部标识符

建立、组织、维护子程序库可以把多个子程序模块统一管理起来。

库文件可看作子程序的集合

库管理程序：LIB.EXE

为解决各模块间标识符的交叉访问，汇编语言提供了2条伪指令：PUBLIC、EXTRN

(1) PUBLIC 标识符，[标识符，... ..]；

是
当前模块中哪些标识符可被其它模块引用的外部标识符

(2) EXTRN 标识符：类型，[标识符：类型，... ..]

当前模块使用的标识符中，哪些已在其它模块中被定义为指定类型的标识符

例.

子程序 stdin.asm

```
PUBLIC stdin
CODE SEGMENT
:
CODE ENDS
END
```

子程序 dispax.asm

```
PUBLIC dispax
DATA SEGMENT
:
CODE SEGMENT
:
END
```

主程序 main.asm

```
EXTRN stdin:FAR, dispax:FAR
STACK SEGMENT
:
STACK ENDS
CODE SEGMENT
ASSUME CS:CODE, SS:STACK
START:
    CALL stdin
    MOV AH,0
    CALL dispax
    MOV AH,4CH
    INT 21H
CODE ENDS
END START
```